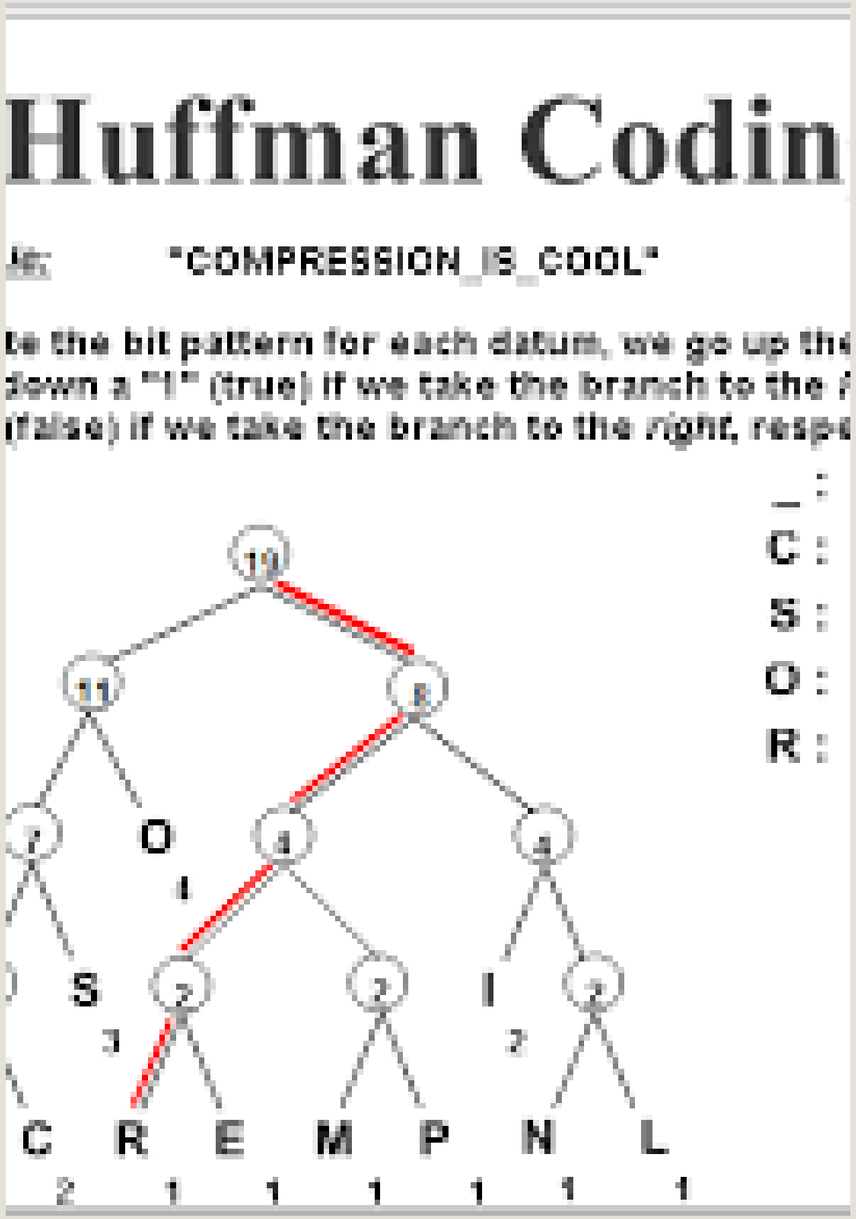
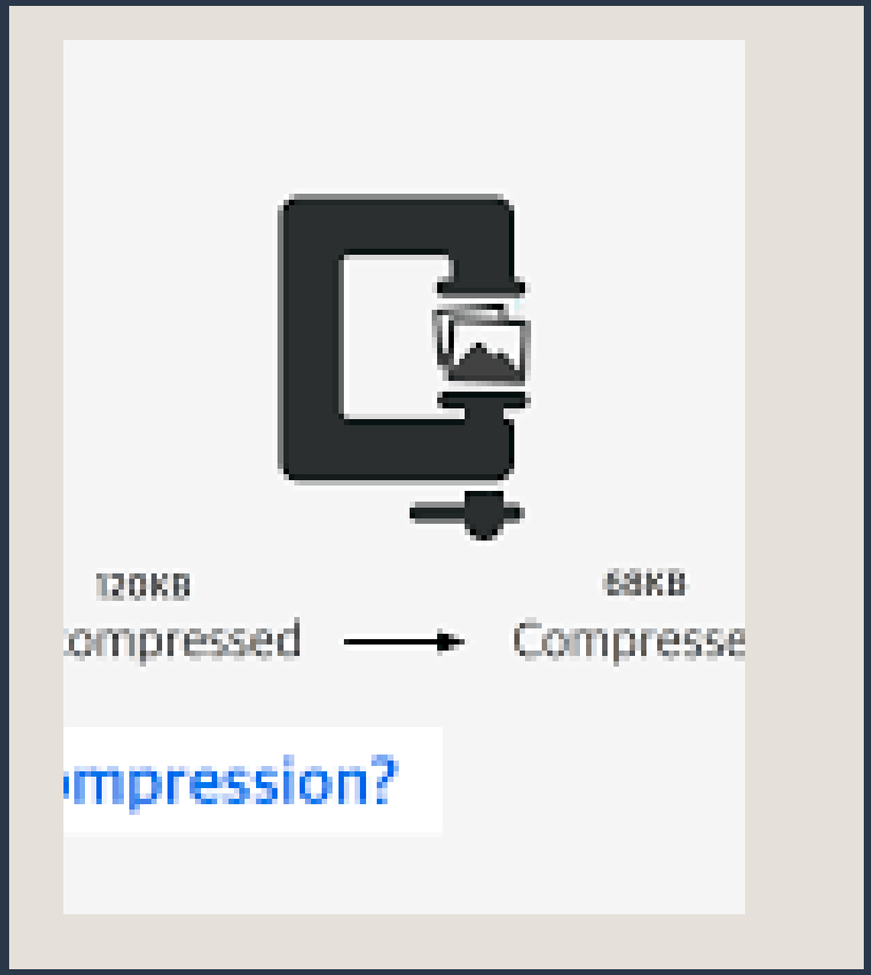
Image compression using huffman coding

Mutharasi - 311124104083
padmashree - 311124104089
punitha - 311124104101





Introduction:

Image compression using Huffman Coding is a method used to reduce video file size. It uses shorter binary codes for frequently used data and longer codes for less used data, helping to save storage space and increase transmission
speed without losing data.

DEFINISION

Huffman Coding is a lossless data compression algorithm used to reduce the size of data by assigning shorter binary codes to frequently used characters and longer codes to less frequent characters.
It is based on the Greedy Technique.

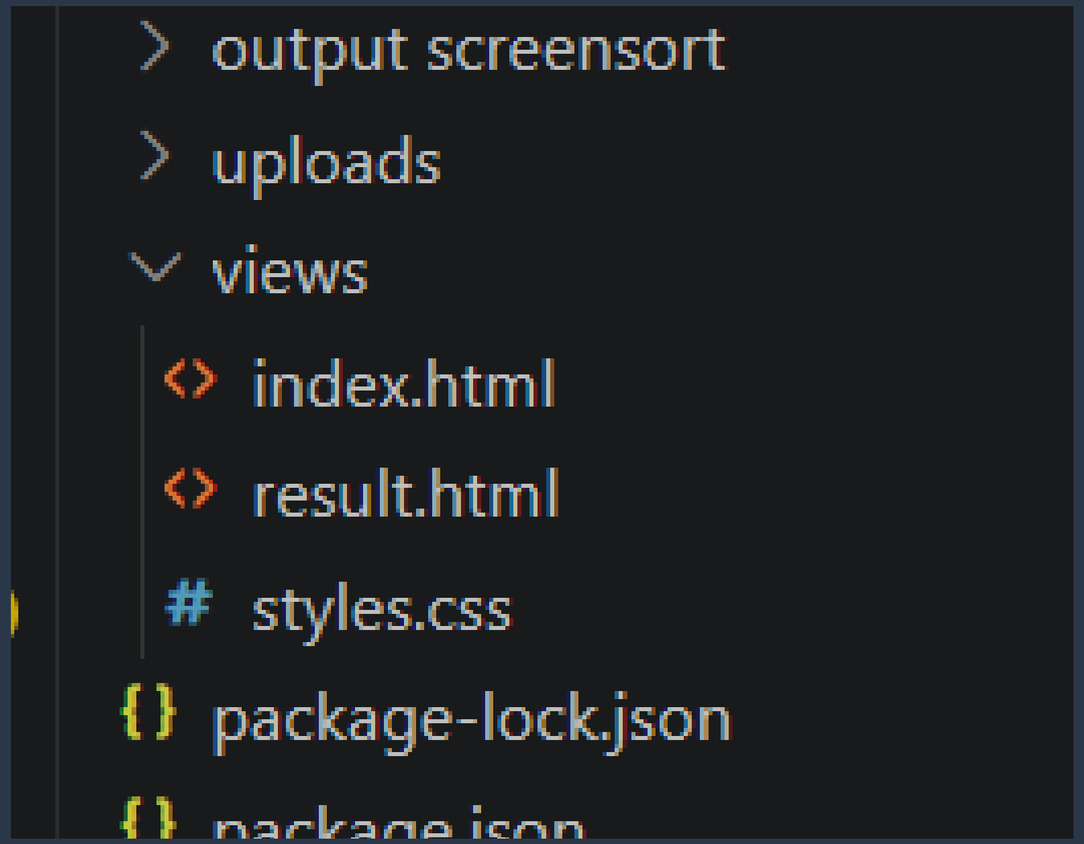
CONCEPT FLOW

STEP 1: UPLOAD / READ IMAGE
STEP 2: CONVERT IMAGE TO PIXEL DATA
STEP 3: REDUCE DATA
STEP 4: COUNT FREQUENCY
STEP 5: BUILD HUFFMAN TREE
STEP 6: GENERATE HUFFMAN CODES
STEP 7: ENCODE IMAGE DATA
STEP 8: STORE / DOWNLOAD COMPRESSED FILE

MINI EXAMPLE

ORIGINAL PIXEL DATA:
SUPPOSE A GRAYSCALE IMAGE HAS PIXEL V
255 255 255 128 128 64
FREQUENCY OF PIXELS:
255 = 3 TIMES
128 = 2 TIMES
64 = 1 TIME
HUFFMAN CODES GENERATED
255 → 0
128 → 10
64 → 11
COMPRESSED DATA
REPLACE PIXEL VALUES WITH HUFFMAN CO
255 255 255 128 128 64
↓
0 0 0 10 10 11
COMPRESSED BINARY:
000101011

INPUT CODE




```
nst multer   = require("multer");
nst fs       = require("fs");
nst path     = require("path");

nst app = express();

 — Serve frontend files ——————————————————
p.use(express.static("views"));

 — Ensure folders exist ——————————————————
nst uploadDir     = path.join(__dirname, "uploads");
nst compressedDir = path.join(__dirname, "compressed");

ploadDir, compressedDir].forEach(dir => {
  if (!fs.existsSync(dir)) fs.mkdirSync(dir, { recursive: true });
;

 — Multer storage ————————————————————————
nst storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename:    (req, file, cb) => cb(null, Date.now() + "-" + file.original
;
nst upload = multer({ storage });

 — MIME type map ————————————————————————
nst MIME_MAP = {
  ".jpg": "image/jpeg", ".jpeg": "image/jpeg",
  ".png": "image/png",  ".gif": "image/gif",
```

INPUT CODE

```
N COMPRESSION


uildHuffmanCodes(data) {
freq = {};
orEach(val => { freq[val] = (freq[val] || 0) + 1; });

gle unique byte edge-case
ject.keys(freq).length === 1) {
nst [val] = Object.keys(freq);
turn { codes: { [val]: "0" }, freq };


des = Object.entries(freq).map(([val, f]) => ({
l: Number(val), f, left: null, right: null


(nodes.length > 1) {
des.sort((a, b) => a.f - b.f || a.val - b.val);
nst left  = nodes.shift();
nst right = nodes.shift();
des.push({ val: null, f: left.f + right.f, left, right });


codes = {};
on generate(node, code = "") {
(node.left === null && node.right === null) {
codes[node.val] = code || "0"; return;
```

```
bitStr) {
= (8 - (bitStr.length % 8)) % 8;
= bitStr + "0".repeat(padBits);
= [];
; i < padded.length; i += 8)
(parseInt(padded.slice(i, i + 8), 2));
edBuffer: Buffer.from(bytes), padBits };



ayout:
 (magic)
 (uint8)
 (uint32 BE)
 (uint32 BE)
ON (UTF-8)
 bits

File({ freq, padBits, origLen, packedBuffer }) {
 = Buffer.from("HUFF");
 = Buffer.from(JSON.stringify(freq), "utf8");
 = Buffer.alloc(9);
nt8(padBits, 0);
nt32BE(origLen,         1);
nt32BE(freqJSON.length, 5);
.concat([magic, header, freqJSON, packedBuffer]);
```

```
= app.listen(3000, () => {
og("╔═══════════════════════════════════════╗");
og("║  ✅ HuffPress running on port 3000    ║");
og("║  http://localhost:3000                ║");
og("╠═══════════════════════════════════════╣");
og("║  Routes loaded:                       ║");
og("║   POST /upload                        ║");
og("║   GET  /image        (preview image)  ║");
og("║   GET  /download-image (save image)   ║");
og("║   GET  /preview      (decompress .huff)║");
og("║   GET  /download     (save .huff)     ║");
og("╚═══════════════════════════════════════╝");


ror", (err) => {
ode === "EADDRINUSE") {
le.error("❌  Port 3000 is already in use!");
le.error("   Run this to kill it, then restart:");
le.error("   Windows: netstat -ano | findstr :3000");
le.error("            taskkill /PID <PID> /F");

le.error("Server error:", err);

xit(1);
```

OUTPUT

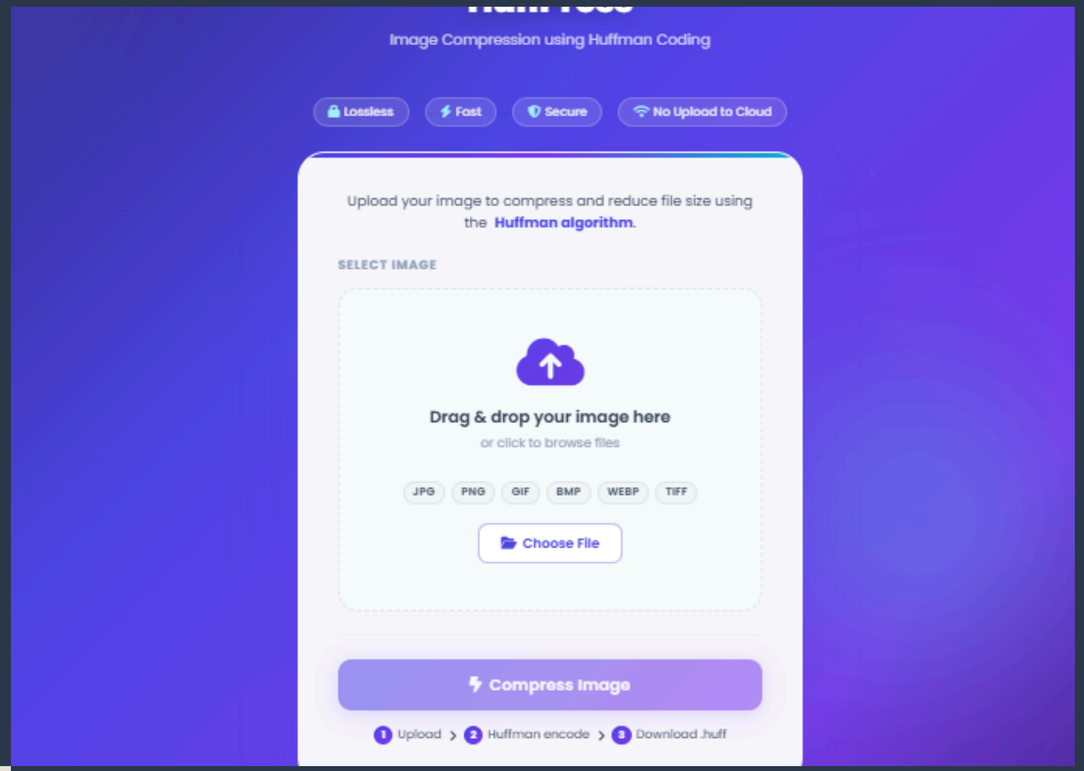


Image Compression using Huffman Coding
Lossless
Fast
Secure
No Upload to Cloud
Upload your image to compress and reduce file size using the Huffman algorithm.
SELECT IMAGE
Drag & drop your image here
or click to browse files
JPG
PNG
GIF
BMP
WEBP
TIFF
Choose File
Compress Image
1 Upload
2 Huffman encode
3 Download .huff

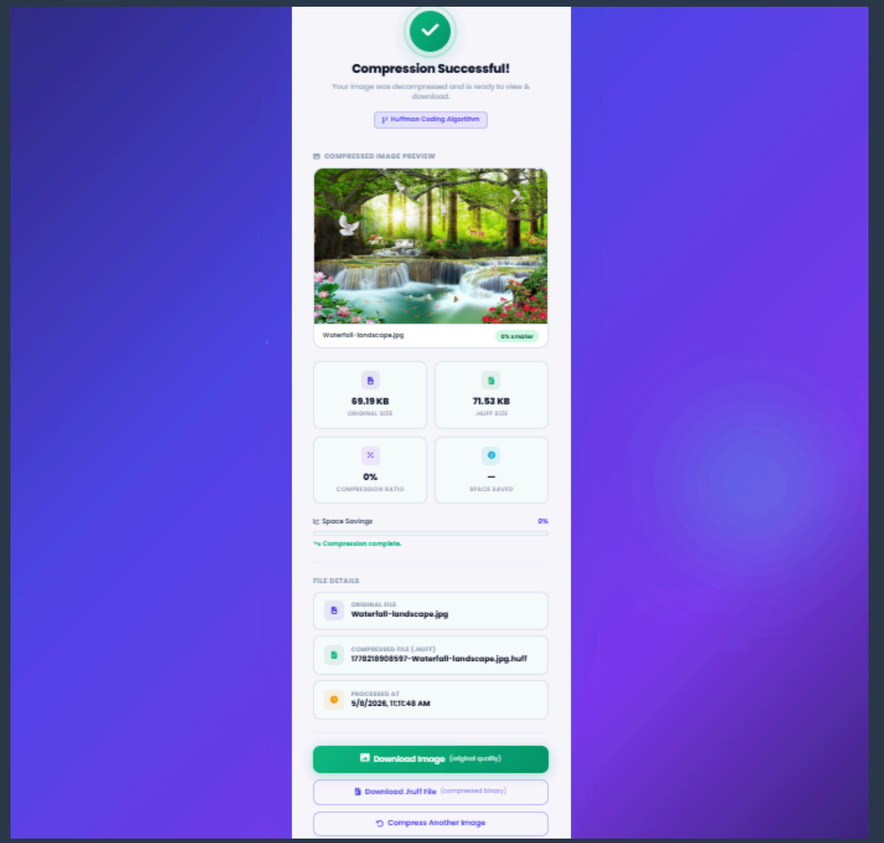

Compression Successful!
Huffman Coding Algorithm
COMPRESSED IMAGE PREVIEW
Waterfall-landscape.jpg
69.19 KB
71.53 KB
0%
Space Savings
Compression complete.
FILE DETAILS
Waterfall-landscape.jpg
Download Image
Download .huff File
Compress Another Image

IMAGE COMPRESSION USING HUFFMAN CODING

CONCEPT FLOW

STEP 1: RECORD AUDIO
STEP 2: CONVERT AUDIO TO DATA
STEP 3: REMOVE EXTRA DATA
STEP 4: COUNT FREQUENCY
STEP 5: BUILD HUFFMAN TREE
STEP 6: GENERATE CODES
STEP 7: COMPRESS AUDIO DATA
STEP 8: STORE / TRANSMIT AUDIO

MINI EXAMPLE

ORIGINAL AUDIO DATA
A A A B B C
FREQUENCY
A = 3
B = 2
C = 1
HUFFMAN CODES
A = 0
B = 10
C = 11
COMPRESSED AUDIO DATA
000101011

ADVANTAGES OF HUFFMAN CODING FOR IMAGE COMPRESSION

1. Lossless Compression

No image data is lost

Original image can be perfectly restored after decompression

2. Reduces File Size

Frequently occurring pixel values get shorter binary codes

Helps reduce storage space and transmission time

3. Efficient for Repeated Data

Works well when images contain repeated or similar pixel values

4. Fast Encoding and Decoding

Compression and decompression are relatively fast

Suitable for real-time applications

5. Simple Algorithm

Easy to understand and implement

Ideal for educational and college projects

6. Improves Storage Efficiency

Smaller files require less disk space

7. Used in Real Compression Systems

Huffman coding is used in formats like:

JPEG

PNG (combined with other methods)

LIMITATIONS OF HUFFMAN CODING FOR IMAGE COMPRESSION

1. LIMITED COMPRESSION RATIO
COMPRESSION EFFICIENCY IS LOWER COMPARED TO ADVANCED METHODS LIKE:
JPEG
2. WORKS POORLY FOR RANDOM DATA
IF PIXEL VALUES ARE EVENLY DISTRIBUTED, HUFFMAN CODING GIVES LITTLE COMPRESSION BENEFIT
3. HUFFMAN TREE OVERHEAD
THE HUFFMAN TREE OR CODE TABLE MUST ALSO BE STORED FOR DECOMPRESSION
THIS INCREASES FILE SIZE SLIGHTLY
4. NO REDUNDANCY REMOVAL
HUFFMAN CODING ONLY ENCODES DATA EFFICIENTLY
IT DOES NOT REMOVE IMAGE REDUNDANCY OR UNNECESSARY INFORMATION
5. SLOWER FOR LARGE IMAGES
BUILDING HUFFMAN TREES FOR VERY LARGE IMAGES CAN INCREASE PROCESSING TIME
6. VARIABLE-LENGTH CODES
CODES HAVE DIFFERENT LENGTHS, MAKING IMPLEMENTATION AND DECODING MORE COMPLEX THAN FIXED-LENGTH ENCODING
7. NOT SUITABLE ALONE FOR MODERN COMPRESSION
MODERN IMAGE FORMATS COMBINE HUFFMAN CODING WITH:
TRANSFORM TECHNIQUES
QUANTIZATION
PREDICTIVE CODING

THANK YOU